CS201 Final

9 January 2024

Olcay Taner YILDIZ

***Abstract*— Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures, (iv) you can use any function given in the data structure.**

I. Question (Algorithm Analysis)

```
1  int algorithm1(int N){
2      if (N == 0)
3          return 0;
4      int sum = 0;
5      for (int i = 0; i < N; i++)
6          for (int j = 0; j < N; j++)
7              sum++;
8      for (int i = N; i > 0; i--)
9          for (int j = 0; j < N; j++)
10             for (int k = 0; k < N; k++)
11                 sum++;
12     for (int i = 0; i < 4; i++)
13         for (int j = 0; j < 2; j++)
14             sum += algorithm1(N / 2);
15     return sum;
16 }
```

What is the time complexity of algorithm1? You **must** construct the recurrence equation and solve it with the master theorem.

II. Question (Graph)

Modify the breadth first search **linked list** implementation such that it will store the length of the shortest paths from the start node in *lengths* parameter.

```
void shortest(int* lengths, int start)
```

At the end of the execution, lengths[i] will show the shortest path length from node $start$ to node i. You may assume that the path length elements are initialized to $vertexCount$ (number of nodes, which should be larger than any possible shortest path) when you call the function.

III. Question (Linked List)

Write the algorithm

```
Node* lastOneWins(int k)
```

in the **LinkedList** class which works as follows:

- Delete every k'th element from the list.
- When you get the end of the list, return to the first element, as if the list is circular.
- Return the remaining node.

Let say the list is 1 2 3 4 5 6, and k = 2, then 2, 4, 6, 3, 1 will be deleted, 5 remains.

IV. Question (Tree)

Write a non-recursive method

```
int* pathList()
```

in the **Tree** class, which returns the keys on the path as an array, where the path is defined by the current parent as follows: If the parent is odd, go left; otherwise go right. The array should contain only that many items not more not less.

V. Question (Sorting)

Suppose arrays representing sets A and B are both sorted. Write a method with **linear complexity** in **MergeSort** class that finds if A is a superset of B.

```
bool isSuperSet(int* A, int* B,
int sizeA, int sizeB)
```

VI. Question (Sorting)

Modify the original selection sort so that

```
int* sortNew(int* A, int* B, int size)
```

(i) it will return the original indexes of the elements as an array and (ii) uses B as a secondary key when keys in A are equal. If original list is 20, 10, 40, 30; after sorting A will be 10, 20, 30, 40, and it will return index array as 1, 0, 3, 2 (10 was at 1., 20 was at 0., 30 was at 3., 40 was at 2. position in the beginning).